

AgenticPlatform

A technical overview — building AI-agent workflows with control, without the complexity

1. The problem

Teams building with AI agents face a trade-off between two kinds of tools. On one side, **visual tools** such as n8n: fast to learn and productive from day one — but coarse-grained, hard to extend, and limiting once the logic gets complex. On the other side, **code frameworks** such as LangChain and LangGraph: extremely powerful and flexible — but with a steep learning curve and a great deal of low-level “plumbing” code to write before anything runs.

The more agents an organisation builds, the more this gap hurts: either you outgrow the visual tool, or every team keeps re-writing the same low-level scaffolding by hand.

2. What AgenticPlatform is

AgenticPlatform is an **open-source framework that sits deliberately in the middle**: the productivity of a declarative tool with the full control of code. Its guiding principle is “*protected, but not caged.*”

It does **not reinvent the engine**. It is built on top of **LangGraph** — a proven, widely used engine for agent workflows — and adds an opinionated, well-structured layer on top. It is also a **self-hosted alternative to the licensed LangGraph Server**: your workflows run on your own infrastructure, behind a standard web API, with no managed runtime to depend on.

3. How it works (the big picture)

The core idea is that the user writes only **two things** — everything else is handled by the framework:

You write	The framework guarantees
Your logic in Python: the agents, the decision points (routers) and the tools.	Wiring between steps, asynchronous execution, error handling, concurrency safety.
Your workflow as a thin YAML file: only the topology — which steps exist and how they connect.	Conversation state, its persistence, and the API that exposes the workflow.

Topology and behaviour are separated. The YAML describes the *shape* of the workflow — a graph of steps; the Python describes *what each step does*. The same diagram a non-technical stakeholder can read maps one-to-one to the executable definition.

A ladder of control. Not every step needs the same effort. The framework offers four levels and you pick the lowest one that does the job — so a simple agent is a handful of lines, and a sophisticated one is still concise:

Level	What you provide	Typical use
L0	Plain Python, no AI	Data steps, validation, reading/writing external systems

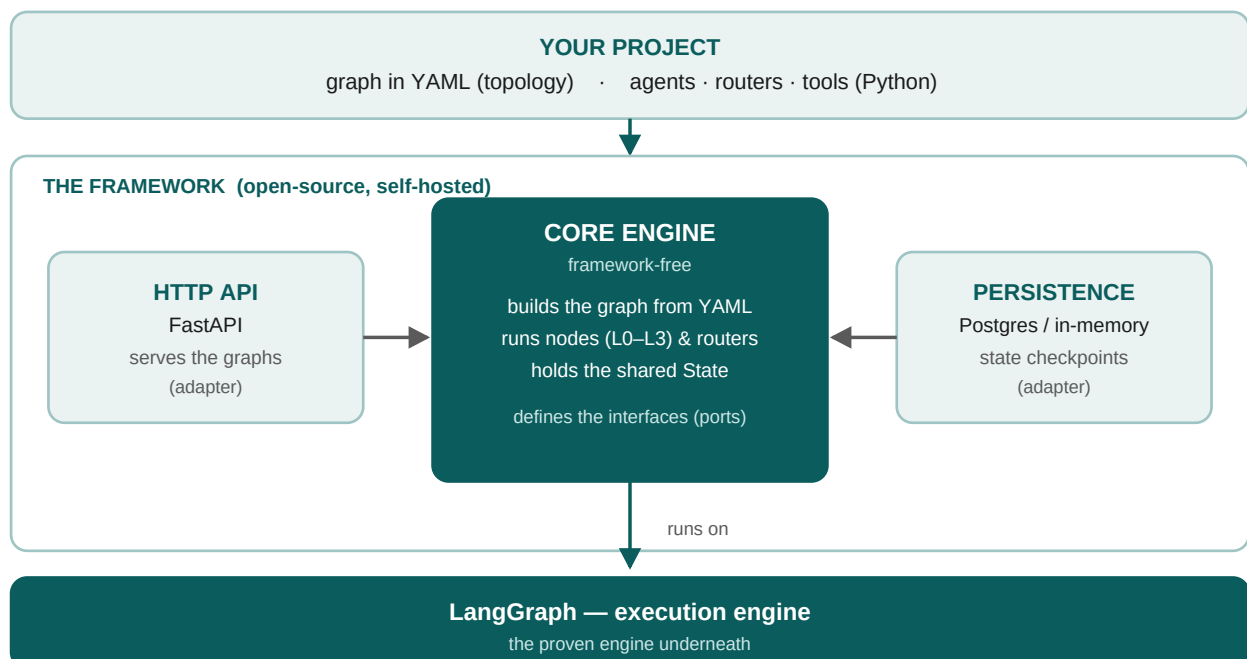
L1	An AI agent declaring only its configuration (model, instructions, tools)	The everyday agent — the level used most
L2	An AI agent plus safety hooks	Validate the input, post-process the output, fall back gracefully on error
L3	Full control of how the AI input and output are shaped	Specialised behaviour, e.g. a quality-check / judge step

Shared state and memory. A shared *state* flows through the workflow: the conversation, plus any runtime data the steps produce. The framework persists this state at every step (optionally in a Postgres database), which gives three business-relevant properties: conversations can **resume** where they left off; every run is **auditable** (you can inspect the state at any past step); and long, looping processes are supported reliably.

Serving. Once defined, a workflow is exposed over a standard HTTP API, on infrastructure you control and with optional authentication — ready to be called by other systems.

A concrete example — a quality-checked answer. A customer asks a question. A first agent (**L1**) drafts an answer using internal tools. A second agent (**L3**) reviews that draft against the original request and either approves it or returns it with specific feedback. A decision point — a **router** — loops back for another attempt until the answer is good enough or a retry limit is reached. A final plain-Python step (**L0**) records the outcome. The whole exchange is a single workflow: a few lines of Python per step and a short YAML describing the loop. It is served as an API, and the entire conversation is persisted — so it can resume, and every attempt can be inspected afterwards.

Architecture at a glance



Ports & Adapters: the core is independent of any web framework or database — the adapters depend on the core, never the other way around. That is what keeps parts swappable and the whole system testable. (Detailed UML class diagrams per subsystem live in the project repository.)

4. Why it is engineered to last

- **Built on a proven engine.** The value is the opinionated layer, not a risky from-scratch engine.
- **Clean architecture.** The core logic is kept independent of any specific web framework or database, so these can be swapped without touching business logic.
- **Guardrails by design.** The API is hard to misuse: common mistakes — a forgotten asynchronous call, unsafe shared state, corrupting the conversation history — are prevented structurally.
- **Testable and maintainable.** Small, single-responsibility components and strict conventions.
- **Open-source and self-hosted.** No vendor lock-in; full ownership of data and runtime.

5. Who it is for, and where it stands

AgenticPlatform is most valuable for **teams and organisations building many agent workflows** — who need both speed and control, and want to keep ownership of their stack. It is **open-source** and at an early but functional stage: it already runs end-to-end (declarative workflows, multi-step agent loops with quality checks, state persistence, HTTP serving, project scaffolding) and has been validated on a real-world use case.

The roadmap focuses on **lowering the entry barrier further** — for example, generating a workflow skeleton directly from a hand-drawn diagram or from an exported n8n workflow — so that going from an idea on a whiteboard to a running, controllable agent system is as short as possible.

6. Where it fits

How AgenticPlatform compares to the two ends of the spectrum it sits between:

	Visual tools (n8n)	AgenticPlatform	Code frameworks (LangGraph / LangChain)
Time to first result	Fast	Fast	Slow
Control & extensibility	Limited	Full — drop into Python anytime	Full
Low-level boilerplate	Low	Low	High
Ownership & hosting	Often SaaS	Self-hosted, open-source	Self-hosted library
Best when	Simple automations	Building many agents, with control	Bespoke, low-level needs

In one sentence: AgenticPlatform lets a team go from a workflow drawn on a whiteboard to a running, production-grade AI-agent system — declaratively and fast, yet with full control and ownership, on a proven engine.

Open-source · built on LangGraph · self-hosted · this document is a high-level overview, not technical documentation.